

AI-Based Landslide Early Warning System — 4-Day Build Roadmap

SIH 2026 | Problem Statement 26001

0. The decision that determines everything else

Pick ONE state/region with good data coverage. Recommend Sikkim or Mizoram — both have published GSI susceptibility data and better DEM/rainfall record coverage than most NER states.

Claim in the pitch: *“We built and validated this for [State] with real data. The pipeline is designed to scale to the rest of NER — here’s the scaling plan.”*

Do NOT claim pan-NER coverage on day 1. A validated single-state model beats an unvalidated regional claim every time a judge asks a follow-up question.

1. What “stands out” actually means here

Most competing teams will do one of these two things:

- A generic dashboard + a black-box ML model with no real data behind it (fails under questioning)
- A rule-based system dressed up as “AI” with no explanation of methodology (fails under questioning)

Your differentiation is honesty + depth in one place, not breadth everywhere:

Differentiator	Why it stands out	Effort
Real susceptibility model on GSI landslide inventory + DEM, with a stated accuracy/validation number	Most teams won’t have a real validated model — they’ll fake it	High
Transparent two-layer risk logic: static susceptibility × dynamic rainfall threshold, explained clearly	Judges can follow the logic, unlike a black-box NN	Medium
“Confidence/uncertainty” shown on the dashboard, not just a risk score	Shows you understand the model’s limits — very few student teams do this	Low, high payoff
Explicit “what’s real vs simulated” slide in the pitch	Pre-empts the exact question that sinks most teams	Zero — just honesty
Offline-first citizen reporting with conflict resolution on sync	Genuinely useful for NER’s connectivity problem, technically nontrivial	Medium
Scaling roadmap slide (data partnerships needed, sensor cost estimate, phased rollout)	Shows product thinking beyond the demo	Low

Skip trying to differentiate on: number of features, UI flashiness, buzzword density (“blockchain,” “quantum,”

“LLM-powered” unless you actually use one meaningfully).

2. Team structure — 6 members, 4 days

Role	Owns	Day 1	Day 2	Day 3	Day 4
A — Data/GIS lead	DEM, slope, GSI landslide inventory, feature engineering	Source & clean DEM + landslide points for chosen state	Compute slope/aspect/drainage layers in QGIS/Python	Hand off features to ML lead; start GIS layer for dashboard	Polish map layers, validate against known landslide zones
B — ML lead	Susceptibility model	Study GSI methodology, set up notebook	Train logistic regression / random forest on features from A	Validate (AUC/ROC, confusion matrix on held-out points), calibrate	Wire model output into API, write “how it works” explanation for pitch
C — Backend lead	API, DB, rainfall integration	Set up FastAPI/Node backend, DB schema	Integrate IMD/open-meteo rainfall API, build risk-escalation rule engine	Build alert trigger logic (SMS/app push), connect to ML lead’s model output	Integration testing, error handling
D — Frontend/dashboard lead	GIS dashboard, heatmap, admin view	Set up React + Leaflet/Mapbox skeleton	Build heatmap layer, risk severity visualization	Build road/village connectivity status view, alert history	Polish UI, responsive check, demo flow
E — Mobile/citizen-reporting lead	Field reporting app, offline sync	Set up React Native/Flutter or PWA skeleton	Geo-tagged photo upload, basic offline storage	Sync-on-reconnect logic, multilingual string setup (2-3 languages minimum)	Test offline→online flow end to end
F — Integration/PM/pitch lead	Glue, demo script, pitch deck, Q&A prep	Coordinate scope decisions, set up shared repo/Figma	Track integration points between A-E, start pitch deck skeleton	Full system dry-run, find gaps, write “real vs simulated” slide	Own the pitch script, run 2-3 mock Q&A rounds with the team

Daily standup: 15 min every morning. Biggest risk in a 4-day build isn't individual skill, it's integration drift — each person's piece not talking to the others by day 4. Person F's job is explicitly to prevent this, not just make slides.

3. Day-by-day milestones (checkpoints, not just tasks)

Day 1 — Foundations. By end of day: state chosen, data sources confirmed accessible (test the IMD API and DEM download NOW, not day 3), all 5 tech skeletons running locally, model plan written down.

Day 2 — Core build. By end of day: susceptibility model trained (even a rough version), backend serving risk data, dashboard showing a static heatmap, mobile app can capture and store a geotagged photo offline.

Day 3 — Integration. By end of day: dashboard pulling live model output, alert logic firing on rainfall-threshold breach, citizen report flowing from mobile → backend → dashboard. This is the day things break — budget the most slack here.

Day 4 — Polish, validation, pitch. Morning: fix integration bugs only, no new features. Afternoon: full run-through, PPT finalized, Q&A prep. Do not write new code after ~2pm on day 4.

Hard rule: if by end of Day 2 the ML model isn't training on real data, fall back to a documented, literature-cited rule-based threshold model and say so explicitly in the pitch. A working honest fallback beats a broken ambitious claim.

4. How to learn what you need, fast

- **GSI landslide susceptibility methodology** — search “GSI landslide susceptibility zonation methodology India” for their published approach; mirror their factor list (slope, aspect, lithology, land use, drainage density, rainfall).
 - **Rainfall intensity-duration thresholds for NE India** — search Google Scholar for published ID-threshold papers specific to the Northeast (there is prior published work on this for parts of India, use it instead of inventing thresholds).
 - **QGIS basics for DEM/slope processing** — QGIS has built-in slope/aspect tools; a few hours of tutorial is enough for what you need.
 - **scikit-learn logistic regression / random forest** — standard, well-documented; don't reach for deep learning, it won't help with this data volume and will be harder to defend.
 - Assign one person (B) to own the model literature so the whole team isn't relearning it separately.
-

5. Jury Q&A — prepare answers to these now, not on stage

Likely question	Weak answer	Strong answer
“Where are your soil moisture sensors?”	“We have IoT sensors” (false, will be	“No dense sensor network exists in NER today — we use satellite soil moisture proxy (SMAP/Sentinel-1) as a substitute, and the architecture supports adding ground sensors when deployed.”

	caught)	
"How accurate is your prediction?"	"Very accurate" / silence	State actual validation metric (AUC, precision/recall on held-out landslide points), and be clear it's <i>susceptibility zonation</i> , not event-time prediction.
"Is this AI or just rules?"	Deflect	"Two layers: a trained ML susceptibility model (static) and a rainfall-threshold trigger (dynamic, rule-based, literature-sourced). We're not claiming the trigger layer is ML — it doesn't need to be."
"How does this scale to all of NER?"	"It just scales"	Name the specific blocker (data coverage per state) and the phased plan to close it.
"What happens with no internet?"	Ignore	Demo the offline capture → sync flow live if possible.
"What's simulated vs real in this demo?"	Avoid the question	Have this pre-answered on a slide before they ask.

6. What "good enough" looks like for a 4-day prototype

You are not shipping a production system. The bar is:

- One real, validated model component
- One clearly-explained rule-based component, not disguised as ML
- End-to-end data flow that works live, even if narrow in scope
- Honest, specific answers about what's real vs. what's a scoped next step

That combination beats a broader but shallower "we cover everything" build almost every time, because it's defensible under questioning — which is where most teams actually lose points.